



MasKot - Product Backlog

Plataforma de nutrición personalizada para mascotas basada en Nutri Co.

Stack: React 19 + Vite + TypeScript + Tailwind CSS + Supabase



Índice de Módulos

1. [Identidad y Landings](#)
2. [Conversión - Nutri-Quiz](#)
3. [Tienda E-commerce](#)
4. [Checkout y Suscripción](#)
5. [Legal y Compliance](#)
6. [Administración Back-office](#)
7. [Módulos Complementarios](#)

Convenciones de Base de Datos (Huanca Stack)

[!IMPORTANT]

Todas las tablas deben seguir las convenciones del Huanca Stack:

- **Naming:** snake_case plural para tablas, snake_case para columnas
- **PKs:** BIGINT GENERATED BY DEFAULT AS IDENTITY o UUID
- **FKs:** [singular_table]_id con ON DELETE explícito
- **ENUMs:** CREATE TYPE [context]_[concept] AS ENUM (nunca TEXT CHECK)
- **Timestamps:** TIMESTAMPTZ siempre (no TIMESTAMP)
- **Checks:** Validar rangos numéricos (>= 0 , BETWEEN , etc.)
- **Fases:** [feature]_logic.sql (MVP) → [feature]_security.sql (RLS/índices)

Módulo 1: Identidad y Landings

HU-1.1: Home Page con Hero Dinámico

User Story: Como visitante, quiero ver una página de inicio atractiva con información clara, para entender el valor de la nutrición personalizada.

Criterios de Aceptación:

- ☐ Hero banner con animación y CTA "Descubre su plan ideal"
- ☐ Comparador costos: comida tradicional vs MasKot
- ☐ Carrusel de testimonios con foto, mascota y resultado
- ☐ Sección "Cómo funciona" en 3 pasos
- ☐ Loading skeleton mientras cargan datos
- ☐ Mobile-first responsive

UI/UX: Hero gradient animado, slider interactivo peso/edad, cards de testimonios con rating, CTA flotante mobile

Frontend: `src/features/home/pages/LandingPage.tsx` + `homePageService.ts`

Supabase (`cms_logic.sql`):

```
CREATE TABLE testimonials (  
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  pet_name TEXT NOT NULL,  
  pet_photo_url TEXT,  
  owner_name TEXT NOT NULL,  
  content TEXT NOT NULL,  
  rating INTEGER NOT NULL CHECK (rating BETWEEN 1 AND 5),  
  is_featured BOOLEAN NOT NULL DEFAULT FALSE,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

```
CREATE TABLE home_page_config (  
  id INTEGER PRIMARY KEY CHECK (id = 1),  
  hero_section JSONB NOT NULL DEFAULT '{} '::jsonb,  
  cost_comparison_section JSONB NOT NULL DEFAULT '{} '::jsonb,  
  how_it_works_section JSONB NOT NULL DEFAULT '{} '::jsonb,  
  cta_banner_section JSONB NOT NULL DEFAULT '{} '::jsonb,  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

HU-1.2: Página "Nosotros"

User Story: Como visitante, quiero conocer la filosofía y equipo de Maskot, para generar confianza antes de comprar.

Criterios de Aceptación:

- ☐ Hero banner con título y subtítulo de la página
- ☐ Sección Misión/Visión con animaciones al scroll
- ☐ Sección explicativa del Motor de IA nutricional
- ☐ Timeline interactivo de historia de la empresa
- ☐ Grid de miembros del equipo con foto, nombre, rol y LinkedIn
- ☐ Galería de certificaciones y avales veterinarios
- ☐ Banner CTA final con llamado a acción
- ☐ Loading skeleton mientras cargan datos dinámicos
- ☐ Mobile-first responsive

UI/UX: Parallax suave en imágenes, cards de equipo con hover effect, timeline vertical en mobile, badges de certificaciones

Frontend: src/features/about/pages/AboutPage.tsx + aboutPageService.ts

Supabase (cms_logic.sql):

```
CREATE TABLE team_members (  
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  full_name TEXT NOT NULL,  
  role TEXT NOT NULL,  
  photo_url TEXT,  
  bio TEXT,  
  linkedin_url TEXT,  
  display_order INTEGER NOT NULL DEFAULT 0 CHECK (display_order >= 0)  
);
```

-- Configuración de Página "Nosotros" (CMS Singleton)

```
CREATE TABLE about_page_config (  
  id INTEGER PRIMARY KEY CHECK (id = 1),  
  -- Sección Hero  
  hero_section JSONB NOT NULL DEFAULT '{}'::jsonb,  
  -- Misión y Visión  
  mission_vision_section JSONB NOT NULL DEFAULT '{}'::jsonb,  
  -- Explicación del Motor de IA  
  ai_engine_section JSONB NOT NULL DEFAULT '{}'::jsonb,  
  -- Timeline de Historia  
  timeline_section JSONB NOT NULL DEFAULT '{}'::jsonb,  
  -- Banner CTA final  
  cta_banner_section JSONB NOT NULL DEFAULT '{}'::jsonb,  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

```
CREATE TABLE certifications (  
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  name TEXT NOT NULL,  
  logo_url TEXT,  
  description TEXT,  
  display_order INTEGER NOT NULL DEFAULT 0 CHECK (display_order >= 0)  
);
```

HU-1.3: Blog Educativo con CMS

User Story: Como visitante, quiero acceder a contenido educativo sobre nutrición de mascotas.

Criterios de Aceptación:

- ☐ Listado de artículos con paginación (12/página)
- ☐ Cards con imagen destacada, título, excerpt y tiempo de lectura
- ☐ Filtros por categoría (`blog_categories`)
- ☐ Búsqueda por título/contenido
- ☐ Vista de artículo individual con slug SEO-friendly
- ☐ Tabla de contenidos (TOC) sticky en desktop
- ☐ Autor del artículo con avatar y nombre
- ☐ Artículos relacionados al final
- ☐ SEO: `meta_title`, `meta_description`, schema markup
- ☐ Estados de artículo: borrador, publicado, archivado
- ☐ Share buttons (WhatsApp, Facebook, Copy link)
- ☐ Loading skeleton mientras cargan datos

UI/UX: Cards con hover effect, sticky TOC en sidebar, breadcrumb de navegación, responsive mobile-first

Frontend: `src/features/blog/pages/BlogListPage.tsx` , `BlogPostPage.tsx` + `blogService.ts`

Supabase (`blog_logic.sql`):

```
CREATE TYPE blog_post_status AS ENUM ('draft', 'published', 'archived');

CREATE TABLE blog_categories (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  name TEXT NOT NULL,
  slug TEXT NOT NULL UNIQUE,
  description TEXT
);

CREATE TABLE blog_posts (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  slug TEXT NOT NULL UNIQUE,
  title TEXT NOT NULL,
  excerpt TEXT,
  content TEXT NOT NULL,
  featured_image_url TEXT,
  category_id BIGINT REFERENCES blog_categories(id) ON DELETE SET NULL,
  author_id UUID REFERENCES profiles(id) ON DELETE SET NULL,
  read_time_minutes INTEGER NOT NULL DEFAULT 5 CHECK (read_time_minutes > 0),
  meta_title TEXT,
  meta_description TEXT,
  status blog_post_status NOT NULL DEFAULT 'draft',
  published_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Módulo 2: Conversión (Nutri-Quiz)

HU-2.1: Flujo Multi-Paso del Quiz

User Story: Como usuario, quiero completar un cuestionario sobre mi mascota para recibir un plan nutricional personalizado.

Criterios de Aceptación:

- ☐ Paso 1: Tipo de mascota (Perro/Gato)
- ☐ Paso 2: Nombre, edad, raza
- ☐ Paso 3: Peso actual y objetivo

- ☐ Paso 4: Nivel de actividad
- ☐ Paso 5: Condiciones especiales (alergias)
- ☐ Paso 6: Preferencias de sabor
- ☐ Barra de progreso visible
- ☐ Guardado en sessionStorage
- ☐ Validación en tiempo real
- ☐ Lógica de saltos: alergias → selector específico

UI/UX: Transiciones slide, ilustraciones por paso, botones grandes mobile, tooltips

Frontend: `src/features/quiz/pages/NutriQuizPage.tsx` + componentes por paso

Supabase (`quiz_logic.sql`):

```

CREATE TYPE pet_type AS ENUM ('dog', 'cat');
CREATE TYPE pet_size_category AS ENUM ('toy', 'small', 'medium', 'large', 'giant');
CREATE TYPE activity_level AS ENUM ('sedentary', 'moderate', 'active', 'very_active');

CREATE TABLE pet_breeds (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  pet_type pet_type NOT NULL,
  name TEXT NOT NULL,
  avg_weight_kg NUMERIC(5, 2) CHECK (avg_weight_kg > 0),
  size_category pet_size_category NOT NULL
);

CREATE TABLE health_conditions (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  name TEXT NOT NULL UNIQUE,
  description TEXT,
  excluded_ingredients TEXT[] NOT NULL DEFAULT '{}'
);

CREATE TABLE flavor_preferences (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  name TEXT NOT NULL UNIQUE,
  icon_url TEXT
);

CREATE TABLE quiz_sessions (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  session_token TEXT NOT NULL UNIQUE,
  user_id UUID REFERENCES profiles(id) ON DELETE SET NULL,
  pet_type pet_type,
  current_step INTEGER NOT NULL DEFAULT 1 CHECK (current_step >= 1),
  answers JSONB NOT NULL DEFAULT '{}'::jsonb,
  started_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  completed_at TIMESTAMPTZ
);

```

HU-2.2: Motor de Recomendación IA

User Story: Como sistema, quiero procesar las respuestas del quiz para generar una recomendación nutricional.

Criterios de Aceptación:

- ☐ Cálculo de requerimiento calórico diario (fórmula RER/MER)
- ☐ Consultar `nutrition_formulas` para obtener fórmulas base
- ☐ Exclusión de ingredientes según `health_conditions.excluded_ingredients`
- ☐ Selección de fórmula según edad, peso y nivel de actividad
- ☐ Generación de score de compatibilidad (0-100) por producto
- ☐ Ajuste de porciones según peso objetivo
- ☐ Tiempo de procesamiento < 2 segundos
- ☐ Manejo de errores con fallback si el motor falla
- ☐ RPC `calculate_pet_nutrition` retorna JSONB con `daily_kcal`, `rer`, `activity_factor`

UI/UX: Animación de "calculando..." mientras procesa, indicador de progreso

Frontend: `src/features/quiz/services/nutritionEngineService.ts`

Supabase (`quiz_logic.sql`):

```

CREATE TABLE nutrition_formulas (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name TEXT NOT NULL,
    base_ingredients TEXT[] NOT NULL DEFAULT '{}',
    suitable_conditions BIGINT[] DEFAULT '{}',
    kcal_per_100g INTEGER NOT NULL CHECK (kcal_per_100g > 0),
    protein_pct NUMERIC(5, 2) NOT NULL CHECK (protein_pct BETWEEN 0 AND 100),
    fat_pct NUMERIC(5, 2) NOT NULL CHECK (fat_pct BETWEEN 0 AND 100),
    fiber_pct NUMERIC(5, 2) NOT NULL CHECK (fiber_pct BETWEEN 0 AND 100)
);

```

```
-- RPC: Calcular nutrición
```

```

CREATE OR REPLACE FUNCTION calculate_pet_nutrition(p_quiz_answers JSONB)
RETURNS JSONB AS $$

```

```
DECLARE
```

```

    v_weight_kg NUMERIC;
    v_rer NUMERIC;
    v_mer NUMERIC;
    v_activity_factor NUMERIC;

```

```
BEGIN
```

```

    v_weight_kg := (p_quiz_answers->>'weight_kg')::NUMERIC;
    -- RER = 70 * (peso^0.75)
    v_rer := 70 * POWER(v_weight_kg, 0.75);
    -- MER = RER * factor actividad
    v_activity_factor := CASE p_quiz_answers->>'activity_level'
        WHEN 'sedentary' THEN 1.2
        WHEN 'moderate' THEN 1.4
        WHEN 'active' THEN 1.6
        WHEN 'very_active' THEN 1.8
        ELSE 1.4

```

```
END;
```

```
    v_mer := v_rer * v_activity_factor;
```

```

    RETURN jsonb_build_object(
        'daily_kcal', ROUND(v_mer),
        'rer', ROUND(v_rer),
        'activity_factor', v_activity_factor
    );

```

```
END;
```

```
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

HU-2.3: Reporte de Diagnóstico

User Story: Como usuario, quiero ver un reporte visual del análisis de mi mascota.

Criterios de Aceptación:

- ☐ Card resumen del perfil: nombre, tipo, raza, peso, actividad
- ☐ Requerimiento calórico diario (`daily_kcal_requirement`)
- ☐ Gráfico radar de necesidades nutricionales (proteína, grasa, fibra)
- ☐ Comparación visual: Comida tradicional vs MasKot
- ☐ Cálculo de ahorro mensual proyectado (`savings_monthly`)
- ☐ Score de nutrición (0-100) (`nutrition_score`)
- ☐ Lista de productos recomendados con % de match
- ☐ Botón "Guardar perfil" → crea registro en `pet_profiles` (requiere login)
- ☐ Botón "Agregar al carrito" directo desde recomendación
- ☐ Compartir reporte por WhatsApp/Email
- ☐ Descargar reporte en PDF
- ☐ Modal de registro/login si usuario no está autenticado

UI/UX: Animación inicial de loading, gráficos con Recharts/Chart.js, highlight del ahorro en verde, confetti al guardar

Frontend: `src/features/quiz/pages/DiagnosticReportPage.tsx` + `petProfileService.ts`

Supabase (`pets_logic.sql`):

```

CREATE TABLE pet_profiles (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  pet_type pet_type NOT NULL,
  breed_id BIGINT REFERENCES pet_breeds(id) ON DELETE SET NULL,
  birth_date DATE,
  weight_kg NUMERIC(5, 2) NOT NULL CHECK (weight_kg > 0),
  target_weight_kg NUMERIC(5, 2) CHECK (target_weight_kg > 0),
  activity_level activity_level NOT NULL DEFAULT 'moderate',
  health_conditions BIGINT[] DEFAULT '{}',
  flavor_preferences BIGINT[] DEFAULT '{}',
  daily_kcal_requirement INTEGER CHECK (daily_kcal_requirement > 0),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE recommendations (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  pet_profile_id BIGINT NOT NULL REFERENCES pet_profiles(id) ON DELETE CASCADE,
  recommended_products JSONB NOT NULL DEFAULT '[]'::jsonb,
  savings_monthly NUMERIC(10, 2) CHECK (savings_monthly >= 0),
  nutrition_score INTEGER CHECK (nutrition_score BETWEEN 0 AND 100),
  generated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

Módulo 3: Tienda (E-commerce)

HU-3.1: Catálogo Personalizado

User Story: Como usuario con mascota, quiero ver productos filtrados según sus necesidades.

Criterios de Aceptación:

- ☐ Listado de productos con cards: imagen, nombre, precio, rating promedio
- ☐ Mostrar solo productos compatibles con perfil activo (suitable_conditions)
- ☐ Excluir productos con ingredientes no aptos (excluded_ingredients)
- ☐ Badge "Recomendado para [nombre mascota]" en productos con alto match
- ☐ Filtros por: categoría (product_categories), precio, tipo (product_type)

- ☐ Ordenar por: relevancia, precio (asc/desc), popularidad (reviews)
- ☐ Vista grid/lista toggle
- ☐ Selector de mascota activa en header (si tiene múltiples `pet_profiles`)
- ☐ Lazy loading de imágenes
- ☐ Skeleton loaders mientras cargan datos
- ☐ Estado vacío si no hay productos compatibles

UI/UX: Cards con hover effect y quick-add button, filtros colapsables en mobile, sticky filters en desktop

Frontend: `src/features/store/pages/CatalogPage.tsx` + `productService.ts`

Supabase (`store_logic.sql`):

```
CREATE TYPE product_status AS ENUM ('draft', 'active', 'out_of_stock', 'discontinued');
CREATE TYPE product_type AS ENUM ('dry_food', 'wet_food', 'topper', 'snack', 'supplement');
```

```
CREATE TABLE product_categories (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  name TEXT NOT NULL,
  slug TEXT NOT NULL UNIQUE,
  parent_id BIGINT REFERENCES product_categories(id) ON DELETE SET NULL,
  display_order INTEGER NOT NULL DEFAULT 0 CHECK (display_order >= 0)
);
```

```
CREATE TABLE products (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  sku TEXT NOT NULL UNIQUE,
  name TEXT NOT NULL,
  slug TEXT NOT NULL UNIQUE,
  description TEXT,
  short_description TEXT,
  price NUMERIC(10, 2) NOT NULL CHECK (price > 0),
  compare_at_price NUMERIC(10, 2) CHECK (compare_at_price > 0),
  images TEXT[] NOT NULL DEFAULT '{}',
  pet_type pet_type,
  product_type product_type NOT NULL,
  category_id BIGINT REFERENCES product_categories(id) ON DELETE SET NULL,
  suitable_conditions BIGINT[] DEFAULT '{}',
  excluded_ingredients TEXT[] DEFAULT '{}',
  weight_options JSONB NOT NULL DEFAULT '[]'::jsonb,
  subscription_available BOOLEAN NOT NULL DEFAULT TRUE,
  stock_quantity INTEGER NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0),
  status product_status NOT NULL DEFAULT 'draft',
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE TABLE product_reviews (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  product_id BIGINT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  rating INTEGER NOT NULL CHECK (rating BETWEEN 1 AND 5),
  comment TEXT,
```

```
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

HU-3.2: Ficha de Producto

User Story: Como usuario, quiero ver información detallada de un producto incluyendo ingredientes y trazabilidad.

Criterios de Aceptación:

- ☐ Galería de imágenes con zoom y swipe en mobile (`products.images[]`)
- ☐ Selector de peso/presentación (`weight_options` JSONB)
- ☐ Toggle: Compra única vs Suscripción con % descuento visible
- ☐ Selector de frecuencia de suscripción (semanal, quincenal, mensual)
- ☐ Tabla nutricional completa (`product_nutrition_facts`)
- ☐ Listado de ingredientes ordenados por porcentaje (`product_ingredients`)
- ☐ Trazabilidad: país de origen y proveedor (`ingredients.origin_country` , `supplier`)
- ☐ Beneficios de cada ingrediente (`ingredients.benefits[]`)
- ☐ Reviews de usuarios con rating y comentario (`product_reviews`)
- ☐ Rating promedio calculado
- ☐ Sección de productos complementarios (cross-sell)
- ☐ Botón "Agregar al carrito" sticky en mobile
- ☐ Breadcrumb de navegación

UI/UX: Tabs (Descripción | Ingredientes | Trazabilidad | Reviews), badge de descuento en suscripción, tooltip de beneficios, thumbnails en galería

Frontend: `src/features/store/pages/ProductDetailPage.tsx` + `productService.ts`

Supabase (`store_logic.sql`):

```

CREATE TABLE ingredients (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    name TEXT NOT NULL UNIQUE,
    description TEXT,
    origin_country TEXT,
    supplier TEXT,
    benefits TEXT[] DEFAULT '{}
);

CREATE TABLE product_ingredients (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    product_id BIGINT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    ingredient_id BIGINT NOT NULL REFERENCES ingredients(id) ON DELETE CASCADE,
    percentage NUMERIC(5, 2) CHECK (percentage BETWEEN 0 AND 100),
    display_order INTEGER NOT NULL DEFAULT 0 CHECK (display_order >= 0)
);

CREATE TABLE product_nutrition_facts (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    product_id BIGINT NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    nutrient TEXT NOT NULL,
    value NUMERIC(10, 2) NOT NULL,
    unit TEXT NOT NULL,
    daily_pct NUMERIC(5, 2) CHECK (daily_pct >= 0)
);

```

HU-3.3: Carrito de Compras

User Story: Como usuario, quiero gestionar mi carrito antes de pagar.

Criterios de Aceptación:

- ☐ Agregar productos al carrito (validar stock_quantity)
- ☐ Quitar productos del carrito
- ☐ Modificar cantidades con validación de stock
- ☐ Toggle compra única/suscripción por item
- ☐ Mostrar subtotal, descuentos aplicados, costo de envío, total
- ☐ Cálculo de envío por distrito (shipping_zones)
- ☐ Sección cross-selling: toppers, snacks, accesorios

- ☐ Input de cupón de descuento con validación (coupons)
- ☐ Mostrar tipo de cupón: porcentaje, monto fijo, envío gratis
- ☐ Validar mínimo de orden y vigencia del cupón
- ☐ Persistir carrito en carts para usuarios logueados
- ☐ Persistir en localStorage para guests
- ☐ Botón "Proceder al checkout"

UI/UX: Sidebar/modal en desktop, página completa en mobile, carousel de sugerencias, input de cupón con feedback visual, resumen sticky

Frontend: src/features/store/pages/CartPage.tsx + cartService.ts

Supabase (store_logic.sql):

```
CREATE TYPE coupon_type AS ENUM ('percentage', 'fixed_amount', 'free_shipping');
```

```
CREATE TABLE shipping_zones (  
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    district TEXT NOT NULL,  
    province TEXT NOT NULL DEFAULT 'Lima',  
    department TEXT NOT NULL DEFAULT 'Lima',  
    shipping_cost NUMERIC(10, 2) NOT NULL CHECK (shipping_cost >= 0),  
    delivery_days INTEGER NOT NULL DEFAULT 3 CHECK (delivery_days > 0),  
    is_active BOOLEAN NOT NULL DEFAULT TRUE  
);
```

```
CREATE TABLE coupons (  
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    code TEXT NOT NULL UNIQUE,  
    type coupon_type NOT NULL,  
    value NUMERIC(10, 2) NOT NULL CHECK (value > 0),  
    min_order NUMERIC(10, 2) CHECK (min_order >= 0),  
    max_uses INTEGER CHECK (max_uses > 0),  
    used_count INTEGER NOT NULL DEFAULT 0 CHECK (used_count >= 0),  
    valid_from TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    valid_until TIMESTAMPTZ,  
    is_active BOOLEAN NOT NULL DEFAULT TRUE  
);
```

```
CREATE TABLE carts (  
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
    user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,  
    session_id TEXT,  
    items JSONB NOT NULL DEFAULT '[]'::jsonb,  
    coupon_id BIGINT REFERENCES coupons(id) ON DELETE SET NULL,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Módulo 4: Checkout y Suscripción

HU-4.1: Flujo de Checkout

User Story: Como usuario, quiero completar mi compra de forma segura.

Criterios de Aceptación:

- ☐ Paso 1: Datos de contacto (registro/login si no autenticado)
- ☐ Paso 2: Selección o creación de dirección de envío (`shipping_addresses`)
- ☐ Autocompletado de distrito con cálculo de costo (`shipping_zones`)
- ☐ Opción "Agregar nueva dirección" con formulario completo
- ☐ Paso 3: Método de envío con costo y días estimados
- ☐ Paso 4: Método de pago (`payment_method` ENUM)
- ☐ Tokenización segura de tarjeta con Culqi/MercadoPago (`payment_tokens`)
- ☐ Mostrar tarjetas guardadas si existen
- ☐ Opción QR para Yape/Plin
- ☐ Aceptación de términos y condiciones (checkbox obligatorio)
- ☐ Validación completa antes de confirmar
- ☐ Crear orden con `order_number` único y `order_items`
- ☐ Página de confirmación con resumen del pedido
- ☐ Email de confirmación automático

UI/UX: Stepper visual de progreso, validación inline, resumen sticky en sidebar, loading state durante pago, animación de éxito

Frontend: `src/features/checkout/pages/CheckoutPage.tsx` + `orderService.ts` , `paymentService.ts`

Supabase (`orders_logic.sql`):

```
CREATE TYPE order_status AS ENUM ('pending', 'confirmed', 'processing', 'shipped', 'delivered',
CREATE TYPE payment_status AS ENUM ('pending', 'paid', 'failed', 'refunded');
CREATE TYPE payment_method AS ENUM ('credit_card', 'debit_card', 'yape', 'plin', 'bank_transfer
```

```
CREATE TABLE shipping_addresses (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  label TEXT NOT NULL DEFAULT 'Casa',
  recipient_name TEXT NOT NULL,
  phone TEXT NOT NULL,
  address_line1 TEXT NOT NULL,
  address_line2 TEXT,
  district TEXT NOT NULL,
  province TEXT NOT NULL DEFAULT 'Lima',
  department TEXT NOT NULL DEFAULT 'Lima',
  postal_code TEXT,
  is_default BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE TABLE orders (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  order_number TEXT NOT NULL UNIQUE,
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE RESTRICT,
  status order_status NOT NULL DEFAULT 'pending',
  subtotal NUMERIC(10, 2) NOT NULL CHECK (subtotal >= 0),
  discount NUMERIC(10, 2) NOT NULL DEFAULT 0 CHECK (discount >= 0),
  shipping_cost NUMERIC(10, 2) NOT NULL DEFAULT 0 CHECK (shipping_cost >= 0),
  total NUMERIC(10, 2) NOT NULL CHECK (total >= 0),
  shipping_address JSONB NOT NULL,
  billing_address JSONB,
  payment_method payment_method NOT NULL,
  payment_status payment_status NOT NULL DEFAULT 'pending',
  notes TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
CREATE TABLE order_items (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  order_id BIGINT NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
  product_id BIGINT NOT NULL REFERENCES products(id) ON DELETE RESTRICT,
  variant_info JSONB,
```

```

quantity INTEGER NOT NULL CHECK (quantity > 0),
unit_price NUMERIC(10, 2) NOT NULL CHECK (unit_price > 0),
is_subscription BOOLEAN NOT NULL DEFAULT FALSE,
subscription_frequency_days INTEGER CHECK (subscription_frequency_days > 0)
);

CREATE TABLE payment_tokens (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  provider TEXT NOT NULL,
  token_id TEXT NOT NULL,
  last_four TEXT NOT NULL,
  card_brand TEXT,
  expires_at TIMESTAMPTZ,
  is_default BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

HU-4.2: Panel de Gestión de Suscripción

User Story: Como suscriptor, quiero gestionar mi suscripción desde mi cuenta.

Criterios de Aceptación:

- ☐ Listado de suscripciones activas del usuario
- ☐ Card por suscripción: producto, cantidad, frecuencia, próximo cobro
- ☐ Indicador de estado (subscription_status ENUM: active, paused, cancelled)
- ☐ Acción: Pausar suscripción (máx 2 meses, guarda pause_until)
- ☐ Acción: Reanudar suscripción pausada
- ☐ Acción: Adelantar próximo envío
- ☐ Acción: Cambiar frecuencia (frequency_days)
- ☐ Acción: Cambiar sabor/variante (variant_info JSONB)
- ☐ Acción: Cambiar dirección de envío (shipping_address_id)
- ☐ Acción: Cambiar método de pago (payment_token_id)
- ☐ Acción: Cancelar con encuesta de motivo (subscription_action log)
- ☐ Historial de acciones en subscription_history
- ☐ Timeline de envíos pasados y próximos

UI/UX: Cards con acciones dropdown, modales de confirmación, feedback instantáneo, encuesta de cancelación multi-step

Frontend: src/features/account/pages/SubscriptionsPage.tsx + subscriptionService.ts

Supabase (subscriptions_logic.sql):

```
CREATE TYPE subscription_status AS ENUM ('active', 'paused', 'cancelled', 'expired');
CREATE TYPE subscription_action AS ENUM ('created', 'paused', 'resumed', 'cancelled', 'frequency

CREATE TABLE subscriptions (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE RESTRICT,
  product_id BIGINT NOT NULL REFERENCES products(id) ON DELETE RESTRICT,
  variant_info JSONB,
  quantity INTEGER NOT NULL DEFAULT 1 CHECK (quantity > 0),
  frequency_days INTEGER NOT NULL DEFAULT 30 CHECK (frequency_days > 0),
  status subscription_status NOT NULL DEFAULT 'active',
  next_billing_date TIMESTAMPTZ NOT NULL,
  shipping_address_id BIGINT NOT NULL REFERENCES shipping_addresses(id) ON DELETE RESTRICT,
  payment_token_id BIGINT NOT NULL REFERENCES payment_tokens(id) ON DELETE RESTRICT,
  pause_until TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE subscription_history (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  subscription_id BIGINT NOT NULL REFERENCES subscriptions(id) ON DELETE CASCADE,
  action subscription_action NOT NULL,
  details JSONB,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Módulo 5: Legal y Compliance (Perú)

HU-5.1: Páginas Legales Estáticas

User Story: Como usuario, quiero acceder a información legal clara.

Criterios de Aceptación:

- ☐ Página de Política de Privacidad (Ley 29733 - Perú)
- ☐ Página de Términos y Condiciones
- ☐ Página de Política de Devoluciones
- ☐ Página de Política de Envíos
- ☐ Contenido dinámico desde `legal_documents`
- ☐ Tipo de documento via `legal_document_type` ENUM
- ☐ Mostrar versión y fecha de publicación (`version` , `published_at`)
- ☐ Navegación por secciones (TOC sticky)
- ☐ Botón de versión imprimible / exportar PDF
- ☐ SEO: meta tags dinámicos por página

UI/UX: Layout limpio con sidebar de navegación, breadcrumb, highlight de secciones al scroll, print-friendly CSS

Frontend: `src/features/legal/pages/LegalDocumentPage.tsx` + `legalService.ts`

Supabase (`legal_logic.sql`):

```
CREATE TYPE legal_document_type AS ENUM ('privacy_policy', 'terms_conditions', 'returns_policy',  
  
CREATE TABLE legal_documents (  
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  type legal_document_type NOT NULL UNIQUE,  
  title TEXT NOT NULL,  
  content TEXT NOT NULL,  
  version TEXT NOT NULL DEFAULT '1.0',  
  published_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

HU-5.2: Libro de Reclamaciones Virtual

User Story: Como usuario, quiero presentar una reclamación formal según normativa peruana.

Criterios de Aceptación:

- ☐ Formulario según formato INDECOPI

- ☐ Selector de tipo: Reclamo o Queja (`complaint_type` ENUM)
- ☐ Datos del consumidor: tipo documento (`document_type` ENUM), número, nombre, email, teléfono
- ☐ Validación de DNI (8 dígitos) y otros documentos
- ☐ Descripción del producto/servicio
- ☐ Detalle de la reclamación
- ☐ Pedido del consumidor
- ☐ Generación automática de número de reclamo (`generate_complaint_number` RPC)
- ☐ Estado del reclamo (`complaint_status` ENUM: pending, in_review, resolved, closed)
- ☐ Generación de PDF con formato oficial (`pdf_url`)
- ☐ Envío de copia por email al consumidor
- ☐ Plazo de respuesta visible (30 días calendario)
- ☐ Página de consulta de estado del reclamo

UI/UX: Formulario multi-step, validación inline, preview antes de enviar, confirmación con animación, descargar PDF

Frontend: `src/features/legal/pages/ComplaintsBookPage.tsx` + `complaintService.ts`

Supabase (`legal_logic.sql`):


```
CREATE TYPE complaint_type AS ENUM ('claim', 'complaint');
CREATE TYPE complaint_status AS ENUM ('pending', 'in_review', 'resolved', 'closed');
CREATE TYPE document_type AS ENUM ('dni', 'ce', 'passport', 'ruc');
```

```
CREATE TABLE complaints (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    complaint_number TEXT NOT NULL UNIQUE,
    type complaint_type NOT NULL,
    status complaint_status NOT NULL DEFAULT 'pending',
    consumer_document_type document_type NOT NULL,
    consumer_document_number TEXT NOT NULL,
    consumer_name TEXT NOT NULL,
    consumer_email TEXT NOT NULL,
    consumer_phone TEXT NOT NULL,
    consumer_address TEXT,
    product_description TEXT NOT NULL,
    complaint_detail TEXT NOT NULL,
    consumer_request TEXT NOT NULL,
    response TEXT,
    responded_at TIMESTAMPTZ,
    pdf_url TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
-- RPC: Generar número de reclamo
```

```
CREATE OR REPLACE FUNCTION generate_complaint_number()
RETURNS TEXT AS $$
DECLARE
    v_year TEXT;
    v_count INTEGER;
BEGIN
    v_year := TO_CHAR(NOW(), 'YYYY');
    SELECT COUNT(*) + 1 INTO v_count
    FROM complaints
    WHERE TO_CHAR(created_at, 'YYYY') = v_year;

    RETURN 'LR-' || v_year || '-' || LPAD(v_count::TEXT, 6, '0');
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

Módulo 6: Administración (Back-office)

HU-6.1: Dashboard de Métricas

User Story: Como administrador, quiero ver métricas clave del negocio.

Criterios de Aceptación:

- ☐ Card de ventas totales: día, semana, mes (sumatorio de `orders.total`)
- ☐ Card de órdenes pendientes (`orders` con `status = 'pending'`)
- ☐ Card de suscripciones activas vs canceladas (`subscriptions.status`)
- ☐ Card de MRR (Monthly Recurring Revenue) calculado
- ☐ Alerta de productos con stock bajo (`stock_quantity < stock_alerts.threshold`)
- ☐ Top 5 productos más vendidos
- ☐ Gráfico de evolución de ventas
- ☐ Filtros por rango de fecha
- ☐ Exportar datos a CSV

UI/UX: Cards con iconos y colores semánticos, gráficos con Recharts, loading skeletons, filtros sticky

Frontend: `src/features/admin/pages/DashboardPage.tsx` + `dashboardService.ts`

HU-6.2: Gestión de Pedidos

User Story: Como administrador, quiero gestionar el ciclo de vida de pedidos.

Criterios de Aceptación:

- ☐ Listado de órdenes con filtros: estado, fecha, cliente, número de orden
- ☐ Ver detalle completo: items, dirección, pago, notas
- ☐ Cambiar estado del pedido (`order_status` ENUM)
- ☐ Registrar cambio de estado en `order_status_history`
- ☐ Agregar tracking de envío (`order_shipments`)
- ☐ Ingresar carrier, número de guía, generar etiqueta
- ☐ Marcar como enviado/entregado (`shipped_at` , `delivered_at`)
- ☐ Procesar devolución con cambio de estado a 'refunded'
- ☐ Agregar notas internas por cambio de estado
- ☐ Búsqueda rápida por `order_number`

UI/UX: Tabla con columnas configurables, modal de detalle, stepper de estado, timeline de historial, inline editing

Frontend: `src/features/admin/pages/OrdersManagementPage.tsx` + `adminOrderService.ts`

Supabase (`orders_logic.sql`):

```
CREATE TABLE order_status_history (  
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  order_id BIGINT NOT NULL REFERENCES orders(id) ON DELETE CASCADE,  
  status order_status NOT NULL,  
  notes TEXT,  
  created_by UUID REFERENCES profiles(id) ON DELETE SET NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

```
CREATE TABLE order_shipments (  
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  order_id BIGINT NOT NULL REFERENCES orders(id) ON DELETE CASCADE,  
  carrier TEXT NOT NULL,  
  tracking_number TEXT NOT NULL,  
  label_url TEXT,  
  shipped_at TIMESTAMPTZ,  
  delivered_at TIMESTAMPTZ  
);
```

HU-6.3: Gestión de Inventario

User Story: Como administrador, quiero controlar el stock de productos.

Criterios de Aceptación:

- ☐ Listado de productos con stock actual (`products.stock_quantity`)
- ☐ Indicador visual de productos bajo umbral (`stock_alerts.threshold`)
- ☐ Registro de movimientos de inventario (`inventory_movements`)
- ☐ Tipos de movimiento: compra, venta, ajuste, devolución, daño (`inventory_movement_type` ENUM)
- ☐ Ajuste manual de stock con motivo obligatorio (`reason`)
- ☐ Referencia a orden relacionada si aplica (`reference_id`)
- ☐ Historial completo de cambios por producto
- ☐ Configurar alertas de stock bajo por producto

☐ Notificaciones por email cuando se alcanza umbral (`notify_emails[]`)

UI/UX: Tabla ordenable, badges de alerta, modal de ajuste manual, historial en sidebar, bulk actions

Frontend: `src/features/admin/pages/InventoryPage.tsx` + `inventoryService.ts`

Supabase (`inventory_logic.sql`):

```
CREATE TYPE inventory_movement_type AS ENUM ('purchase', 'sale', 'adjustment', 'return', 'damage');
```

```
CREATE TABLE inventory_movements (  
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  product_id BIGINT NOT NULL REFERENCES products(id) ON DELETE CASCADE,  
  type inventory_movement_type NOT NULL,  
  quantity INTEGER NOT NULL,  
  reason TEXT,  
  reference_id TEXT,  
  created_by UUID REFERENCES profiles(id) ON DELETE SET NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

```
CREATE TABLE stock_alerts (  
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  product_id BIGINT NOT NULL REFERENCES products(id) ON DELETE CASCADE,  
  threshold INTEGER NOT NULL CHECK (threshold >= 0),  
  notify_emails TEXT[] NOT NULL DEFAULT '{}',  
  is_active BOOLEAN NOT NULL DEFAULT TRUE  
);
```

HU-6.4: Panel de Costos de Insumos

User Story: Como administrador, quiero actualizar costos de insumos para reflejar el ahorro real.

Criterios de Aceptación:

- ☐ Listado de ingredientes con costo actual (`ingredient_costs`)
- ☐ Editar costo por kg (`cost_per_kg`) con moneda (`currency`)
- ☐ Historial de costos con fecha efectiva (`effective_date`)
- ☐ Registro de quién hizo el cambio (`created_by`)
- ☐ Listado de precios de competencia (`competitor_prices`)

- ☐ Registrar marca, precio, peso por tipo de producto
- ☐ Recalcular automáticamente el ahorro mostrado al usuario
- ☐ Última actualización visible en cada registro

UI/UX: Tablas editables inline, comparador visual MasKot vs competencia, gráficos de tendencia de costos

Frontend: `src/features/admin/pages/CostsPanel.tsx` + `costsService.ts`

Supabase (`admin_logic.sql`):

```
CREATE TABLE ingredient_costs (  
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  ingredient_id BIGINT NOT NULL REFERENCES ingredients(id) ON DELETE CASCADE,  
  cost_per_kg NUMERIC(10, 2) NOT NULL CHECK (cost_per_kg > 0),  
  currency TEXT NOT NULL DEFAULT 'PEN',  
  effective_date TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  created_by UUID REFERENCES profiles(id) ON DELETE SET NULL  
);
```

```
CREATE TABLE competitor_prices (  
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,  
  product_type product_type NOT NULL,  
  brand TEXT NOT NULL,  
  price NUMERIC(10, 2) NOT NULL CHECK (price > 0),  
  weight_kg NUMERIC(5, 2) NOT NULL CHECK (weight_kg > 0),  
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Módulo 7: Complementarios

HU-7.1: Autenticación y Perfiles

User Story: Como usuario, quiero crear cuenta y gestionar mi perfil.

Criterios de Aceptación:

- ☐ Registro con email/contraseña (Supabase Auth)
- ☐ Login con email/contraseña

- ☐ Login social con Google OAuth
- ☐ Flujo de recuperación de contraseña por email
- ☐ Verificación de email obligatoria
- ☐ Crear perfil automáticamente al registrarse (`profiles`)
- ☐ Asignar rol 'user' por defecto (`user_roles`)
- ☐ Página de perfil con datos personales editables
- ☐ Subir avatar (`avatar_url`)
- ☐ Agregar tipo y número de documento (`document_type` , `document_number`)
- ☐ Gestionar múltiples direcciones de envío (`shipping_addresses`)
- ☐ Ver mascotas guardadas (`pet_profiles`)
- ☐ Ver historial de pedidos (`orders`)

UI/UX: Formularios con validación inline, avatar con crop, tabs en perfil (Datos | Direcciones | Mascotas | Pedidos), skeleton loaders

Frontend: `src/features/security/` (Guards, Services, Pages) + `src/features/account/`

Supabase (`core_logic.sql`):

```

-- Sigue el patrón estándar de profiles/roles/user_roles
CREATE TABLE profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  username TEXT UNIQUE,
  full_name TEXT NOT NULL,
  phone TEXT,
  document_type document_type,
  document_number TEXT,
  avatar_url TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE roles (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  name TEXT UNIQUE NOT NULL,
  description TEXT
);

INSERT INTO roles (id, name, description) VALUES
  (1, 'user', 'Usuario estándar'),
  (2, 'admin', 'Administrador');

CREATE TABLE user_roles (
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  role_id BIGINT NOT NULL REFERENCES roles(id) ON DELETE CASCADE,
  assigned_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  PRIMARY KEY (user_id, role_id)
);

```

HU-7.2: Sistema de Notificaciones

User Story: Como usuario, quiero recibir notificaciones sobre pedidos y suscripciones.

Criterios de Aceptación:

- ☐ Email: Confirmación de pedido (al crear orden)
- ☐ Email: Pedido enviado con link de tracking
- ☐ Email: Pedido entregado
- ☐ Email: Recordatorio de cobro de suscripción (3 días antes)

- ☐ Email: Cobro de suscripción exitoso
- ☐ Email: Fallo en cobro de suscripción
- ☐ Preferencias configurables por usuario (notification_preferences)
- ☐ Toggle: emails de pedidos (email_orders)
- ☐ Toggle: emails de suscripción (email_subscription)
- ☐ Toggle: emails de marketing (email_marketing)
- ☐ Registro de todos los envíos en notification_logs
- ☐ Estado del envío: pending, sent, failed (notification_status ENUM)
- ☐ Canal: email, push, sms (notification_channel ENUM)

UI/UX: Página de preferencias con toggles, historial de notificaciones enviadas, badges de estado

Frontend: src/features/account/pages/NotificationPreferencesPage.tsx + notificationService.ts

Supabase (notifications_logic.sql):

```
CREATE TYPE notification_channel AS ENUM ('email', 'push', 'sms');
CREATE TYPE notification_status AS ENUM ('pending', 'sent', 'failed');
```

```
CREATE TABLE notification_preferences (
  user_id UUID PRIMARY KEY REFERENCES profiles(id) ON DELETE CASCADE,
  email_orders BOOLEAN NOT NULL DEFAULT TRUE,
  email_subscription BOOLEAN NOT NULL DEFAULT TRUE,
  email_marketing BOOLEAN NOT NULL DEFAULT FALSE,
  push_enabled BOOLEAN NOT NULL DEFAULT FALSE,
  sms_enabled BOOLEAN NOT NULL DEFAULT FALSE
);
```

```
CREATE TABLE notification_logs (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  type TEXT NOT NULL,
  channel notification_channel NOT NULL,
  subject TEXT,
  content TEXT,
  status notification_status NOT NULL DEFAULT 'pending',
  sent_at TIMESTAMPTZ
);
```


HU-7.3: Programa de Referidos

User Story: Como usuario, quiero referir amigos y obtener beneficios.

Criterios de Aceptación:

- ☐ Generar código único de referido por usuario (`referral_codes.code`)
- ☐ Compartir código vía WhatsApp, Email, copiar al clipboard
- ☐ Descuento automático para el referido (`referred_discount`)
- ☐ Crédito/recompensa para el referidor (`referrer_reward`)
- ☐ Vincular referido con orden (`referrals.order_id`)
- ☐ Estados de referido: pending, completed, expired (`referral_status` ENUM)
- ☐ Dashboard personal de referidos con contador (`uses_count`)
- ☐ Lista de referidos con estado y recompensa obtenida
- ☐ Límite mensual configurable (regla de negocio)

UI/UX: Card de código destacado, botones de share nativos, progress bar hacia próxima recompensa, tabla de historial

Frontend: `src/features/account/pages/ReferralsPage.tsx` + `referralService.ts`

Supabase (`referrals_logic.sql`):

```

CREATE TYPE referral_status AS ENUM ('pending', 'completed', 'expired');

CREATE TABLE referral_codes (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  code TEXT NOT NULL UNIQUE,
  uses_count INTEGER NOT NULL DEFAULT 0 CHECK (uses_count >= 0),
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE referrals (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  referrer_id UUID NOT NULL REFERENCES profiles(id) ON DELETE RESTRICT,
  referred_user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE RESTRICT,
  order_id BIGINT REFERENCES orders(id) ON DELETE SET NULL,
  referrer_reward NUMERIC(10, 2) NOT NULL DEFAULT 0 CHECK (referrer_reward >= 0),
  referred_discount NUMERIC(10, 2) NOT NULL DEFAULT 0 CHECK (referred_discount >= 0),
  status referral_status NOT NULL DEFAULT 'pending',
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

HU-7.4: SEO y Analytics

User Story: Como negocio, quiero optimizar visibilidad y medir rendimiento.

Criterios de Aceptación:

- ☐ Meta tags dinámicos por página (title, description, og:image)
- ☐ Sitemap.xml generado automáticamente
- ☐ Robots.txt configurado correctamente
- ☐ Schema markup para productos (Product)
- ☐ Schema markup para organización (Organization)
- ☐ Schema markup para FAQs (FAQPage)
- ☐ Integración Google Analytics 4
- ☐ Integración Facebook Pixel
- ☐ Google Tag Manager configurado
- ☐ Eventos de conversión: registro, quiz completado, add to cart, purchase
- ☐ Tracking de suscripciones activadas

UI/UX: N/A (backend/infraestructura)

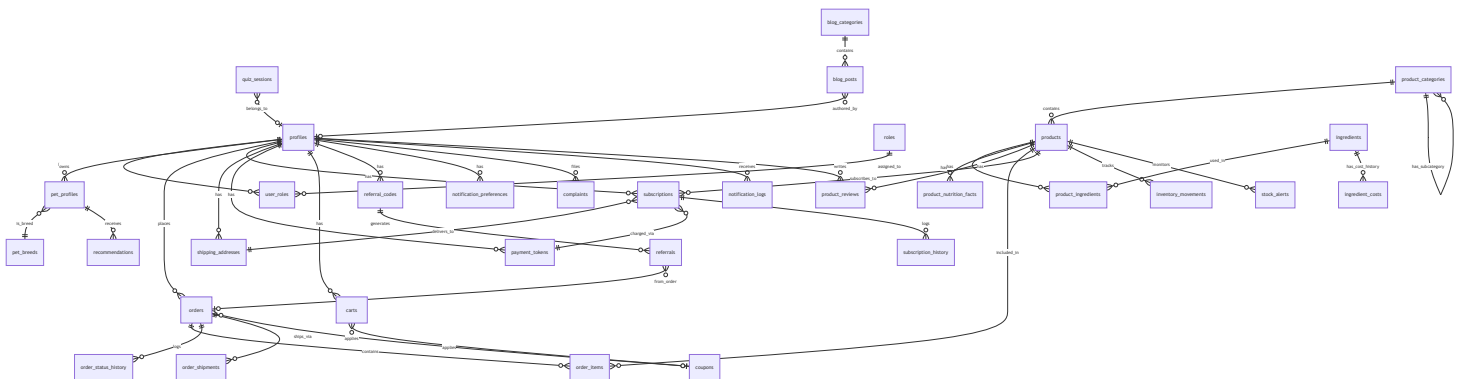
Frontend: Componente `SEOHead.tsx` reutilizable, hooks para tracking events

Estructura de Archivos SQL

supabase/

```
├─ core_logic.sql      # profiles, roles, user_roles
├─ cms_logic.sql       # home_page_config, testimonials, team, blog
├─ quiz_logic.sql      # quiz_sessions, pet_breeds, conditions, formulas
├─ pets_logic.sql      # pet_profiles, recommendations
├─ store_logic.sql     # products, categories, reviews, carts, coupons
├─ orders_logic.sql    # orders, order_items, shipping, payments
├─ subscriptions_logic.sql # subscriptions, history
├─ legal_logic.sql     # legal_documents, complaints
├─ inventory_logic.sql # movements, alerts
├─ notifications_logic.sql # preferences, logs
├─ referrals_logic.sql # codes, referrals
└─ admin_logic.sql     # ingredient_costs, competitor_prices
```

Modelo de Datos - ER Diagram





Priorización MoSCoW

Must Have (MVP)

- HU-1.1: Home Page
- HU-2.1, 2.2, 2.3: Nutri-Quiz completo
- HU-3.1, 3.2, 3.3: Catálogo y Carrito
- HU-4.1: Checkout
- HU-5.1, 5.2: Legal + Libro Reclamaciones
- HU-7.1: Autenticación

Should Have

- HU-4.2: Gestión de Suscripción
- HU-6.1, 6.2, 6.3: Admin básico
- HU-7.2: Notificaciones

Could Have

- HU-1.2, 1.3: Nosotros y Blog
- HU-6.4: Panel de Costos
- HU-7.3: Referidos
- HU-7.4: SEO/Analytics

MasKot - Pet Nutrition Platform

Última actualización: 2026-01-30